tegus by AlphaSense | Expert Insights

COMPANY DEEP-DIVE

# Head of Forward Deployed Engineer at Cresta Believes Proprietary AI Frameworks Are Essential for Enterprise Success

DATE PUBLISHED
16 Sep 2026

INTERVIEW DATE
18 Jun 2026

EXPERT PERSPECTIVE
Consultant

ANALYST PERSPECTIVE
Investor-Led (Buy-Side)

PRIMARY COMPANY
LANGCHAIN

OTHER COMPANIES
ELEVENLABS OPENAI

SMART SUMMARY

## Summary

The interview covered the design, deployment, and operational considerations of enterprise AI agents, including the rationale for building a proprietary framework, approaches to evaluation and cost optimization, and reactions to features of a developer-focused agent deployment platform.

EXPERT BACKGROUND

- Head of Forward Deployed Engineering at an enterprise SaaS company specializing in AI agents for customer experience, with deep experience building and deploying AI agent frameworks for large-scale call center operations.

EXPERT SENTIMENT

- Believes that building a custom AI agent framework is necessary to meet enterprise requirements for low latency, reliability, and customization, rather than relying on open source or commercial frameworks.
- Views cost optimization and model selection as critical, with a strong focus on balancing performance, customer requirements, and operational margins.
- Is **skeptical about the practicality of automated model routing and continuous swapping of models in production environments**, emphasizing the need for stability and control.

TAKEAWAYS

- **The company operates a SaaS platform for AI agents, primarily serving enterprise customers in sectors like airlines, banking, and home security, focusing on improving customer experience and reducing operational costs**.
- A proprietary agent orchestration framework was developed in-house due to the need for low latency, vendor agnosticism, robust guardrails, and deep customization, which existing frameworks like LangChain could not provide.
- The framework supports modular agent design (umbrella and sub-agents) and allows rapid deployment of new use cases via configuration, not code rewrites.
- Rigorous evaluation and testing are integral, often taking longer than initial agent development, with extensive use of simulation, synthetic customers, and automated evaluators to ensure quality and compliance.
- Cost optimization is managed by careful model and voice engine selection, detailed tracking of resource usage, and leveraging open source models for non-critical tasks, but main agents use frontier models for reliability.

AlphaSense Assistant is AI Generated

## Table of Contents

Introduction and Overview

Harness Framework Development

Agent Evaluation and Optimization

Cost Management Strategies

Industry Trends and Innovations

## Expert Bio

Head of Forward Deployed Engineer at Cresta. The expert can speak to all screening questions. Head of Forward Deployed Engineer at Cresta. The expert is responsible for customer-facing AI agent deployments in the contact center and customer service space. Their role involves overseeing the strategy, execution, and scaling of high-impact AI agent implementations, where FDEs work directly with enterprise customers to develop, configure, integrate, deploy, and optimize Cresta's AI agents using the company's platform, while collaborating closely with internal software, machine learning, and product teams to ensure successful outcomes, drive continuous platform



©2026, AlphaSense, Inc. All Rights Reserved. AlphaSense is a service mark of AlphaSense, Inc. All other trademarks mentioned belong to their respective owners.

improvements, and maximize customer ROI. Prior to Cresta, the expert was the Co-Founder & Head of Product at Timeplus, leaving in August 2025. They were responsible for defining and driving the overall product vision, strategy, and roadmap for Timeplus's unified real-time streaming analytics platform, including its core engine, Proton, and enterprise features focused on stream processing, real-time data pipelines, and analytics for telemetry, AI, and mission-critical applications.

## Interview Transcript

Client 00:00:00

Thanks for taking the time. Super appreciate it. Essentially, we've been learning more about the AI agent harnesses space. We're essentially learning more about the opportunity, just trying to assess the market, get a thesis on it, talking to a few experts. Towards the second half of the call, I'd love to get your perspective on the company as well. I'd love to share with you some of the features that they have and the value proposition they have for harnesses, and get your overall reaction to that. First, if you could just start with a quick overview of your experience and the company that you work at currently.

Expert 00:00:33

Yeah, sure. I am the head of AI agents FDE at Cresta. FDE is Full Deployed Engineering. Our company, right now, is really focusing on using AI to improve the customer experience, can be voice or can be chat or email or SMS, but leveraging enterprise AI technology to either reduce cost or improve the customer satisfaction, do upsell, minimize the training, and we do not have to worry about hire or train or retain those human agents, because after all, being a human agent is not really an easy job or it is not a fun job, so the attrition rate is very high. Now, the AI is almost ready. We can leverage the best frontier model together with the right voice engine to pick up some of the use cases.

Cresta, as you may know, it is founded nine years ago. It is always focusing on the call center. In the past, there is a lot of software for human agents. For example, let them know what is the checklist and what is the past conversation this customer calling for. Also, help them to do the search. The recent two years, Cresta really invest a lot on the AI agents. Building AI agent is not easy. That is the reason why we have this team, FDE. I am happy to be the lead of that team to a set of AI experts to leverage different harness framework, and also knowing our platform inside out and work very closely with clients to build the AI agent solutions.

For example, we have customers such as Alaska Airlines and many national parks in the U.S., a lot of banks, home security, something like that. Building enterprise AI solution is our key value proposition, and we do evaluate many harness framework on the market, and we also actually build our own framework to have extra controls. We can maybe start from here.

---

Client 00:02:38

Makes sense. Just to get an understanding as well within the company, what does a production-ready agent look like internally? What bar does it have to clear before you can ship an agent to prod?

Expert 00:02:50

Production AI agent, meaning that is when the guests of our customers, say this is an airline or this is a bank, their guests call the phone number, it will hit a call center and it will, based on their use cases, they can talk to our agents without waiting for a couple of minutes or even one hour in a queue for a human agent. If those calls are routed to our AI agents and we are taking thousands of calls every single day, then this is already in production.

We might not be able to cover all these cases, but production AI agents really making sure that it follows the business standard operating procedure, SOP, and then getting their latest knowledge base, calling the right API. It's not just a read-only, more like FAQ. It can also, for example, make decisions, booking, rebooking, cancellation. It's almost like a human agent, but it just burn the token not to have to hire anyone.

---

Client 00:03:49

An agent that can take action, is accurate, and doesn't have high latency. It's quick to act. Is that pretty much it?

Expert 00:03:58

Yeah, okay. We certainly needed to disclose the guest, you are talking to an AI agent. We don't want to cheat, since the latency is so low and the conversation can be very natural, also they have the right API access, so they can do many things human agent can do. There's always a case, no matter customer request to talk to human agents or there are certain cases is out of scope for this AI agent, we can always transfer to the human.

---


©2026, AlphaSense, Inc. All Rights Reserved. AlphaSense is a service mark of AlphaSense, Inc. All other trademarks mentioned belong to their respective owners.

Client 00:04:28

Perfect. I do want to get into a few harness-related questions. The first one being, let's go at the beginning of building a harness. When you start building a harness for a new use case, what does day one look like? Do you start from a framework like LangChain and as you know, I saw in the screeners as well that you mentioned using Pydantic, and custom-built frameworks as well. I'd love to understand a little bit more how that works. What platforms do you use? Do you copy a previous harness and start from there? Do you completely have to build from scratch every time? How does that work in terms of harness building?

Expert 00:05:07

Sure. At Cresta, we are more like a SaaS Software as a Service company. We do not really have any on-premise deployments. In many cases, we operate on AWS, and it is a multi-tenant environment, meaning we do have a cluster in different AWS regions. For example, this is U.S., this is Canada or in Europe. We can set up the cluster and other cluster. Each cluster might support multiple customers, and the data is logically isolated. We are a SaaS company. We are a product company. We do provide this more like AI agent platform or AI agent infrastructure. We implement, for example, this is airline, we implement the AI agent. That AI agent is running on our own infrastructure. We can integrate with their call center system.

We can still reuse the call center the customer used to use, but instead of trying to connect to a human agent, it will connect to us, the Cresta platform. We do not really more like fork our platform for every single customer. It is running on the same AI agent platform, but we do have a Docker image or certain building block to host that custom code for each customer. Every single customer, there is one or multiple agents, and each agent can have sub-agents. We can talk about that later on. All those prompts, all those function tool calls, guardrails, and other settings are isolated per user, and they are leveraging the same platform capability.

This agent orchestration framework is actually built by us. We are aware of tools such as LabKit or LangChain, but in the end, we decided to build our own orchestration framework. We can get inspired by open source, other framework, but we choose not to use any of them out of the box.

---

Client 00:07:11

I think I would like to double-click on this. I understand that you are more of a SaaS platform centralized for agent, but to a certain degree, you did create agents as well, like the customer experience one. I want to understand a little bit more what you mean by this own framework that you guys have created. You evaluated different framework options, and you mentioned that using Pydantic, LangChain is different, but Pydantic as well. I just want to reconcile and understand a little bit more what that means. You use Pydantic, but you also use a custom-built agent framework. What is part of that framework, and what was part of that decision process to essentially build your own as opposed to maybe going to a LangChain or something like that?

Expert 00:07:55

Sure. Again, almost half of the engineers at Cresta, they came in from Google Call Center AI, CCAI team. They are very technical. They know a lot of industrial patterns, and also those open source ones, for example, the LangChain. The reason why we decided to build our own agent, the decision was almost made two years ago, and we keep revisiting that decision, making sure this is right. The main reason is there is always a lot of enterprise features we want to build into the framework. For example, it needs to be very low latency. It should minimize the hallucination. It should be vendor agnostic and also have a lot of, like for example, guardrails or those customization.

Using a framework such as LangChain actually can be a good enough demo, but **you lose the control** regarding how you can minimize the latency, how you can introduce an extra step. Unless you have to fork it. If you fork it, then **it is almost hard to, almost impossible to maintain and keep up with the upstream**. There are so many extra stuff we want to do, but it is not available in any open source or even commercial framework. We decided to build our own framework, and that framework, for example, we can support, for example, ElevenLabs or Cartesia, different voice engine.

Also, we have other system building. For example, if we realize ElevenLabs or Cartesia is not fast enough, or maybe they have a cloud outage, or **the performance is downgraded**, our framework can automatically fall back to other systems. For example, we do have Deepgram, the other voice engine running on our own DS GPU. **This is actually as our fallback solution,** and we only have one chance to fall back. Otherwise, if you move from A to B, B to C, **the experience to the end user will be terrible**. We decided to say if we are using ElevenLabs for the main voice engine, but in the middle of it, for example, all those first byte to audio, all those metrics is not meeting our criteria, it will automatically fall back to Deepgram. Those things can be easily enhanced by our own framework.

---

Client 00:10:16

If I recap, if I put this in my own words, essentially, you created your own framework because you have a bunch of highly technical and who have domain-specific understanding of customer experience, engineers that come from Google and companies like as such. It was just simpler for you to, with that technical expertise plus domain expertise, to create your own agent framework for the agents that you are offering to your customers. Is that, in a nutshell, what it is?


©2026, AlphaSense, Inc. All Rights Reserved. AlphaSense is a service mark of AlphaSense, Inc. All other trademarks mentioned belong to their respective owners.

Expert 00:10:42
Yeah, that's right. Yep.

---

Client 00:10:46
Perfect. That makes sense. Maybe my question then, in that case, this seems like you essentially have built custom harnesses for your agents. How do you make sure that you don't over-engineer or over-scaffold your agents? Given that they are completely customized, how do you make sure that this doesn't happen? When you want to launch or when you want to start a new agent, let's say you have another use case in customer experience, do you start completely from scratch with your engineers, and do they start a harness from zero? Is there a way that you have templates, harnesses through your framework that you have developed in-house, and that you can just duplicate those frameworks and/or duplicate those templates and start building from there?

Expert 00:11:29
Sure. It will be a long answer. First of all, there is a lot of harness framework outside. For example, even Claude Code is a harness framework in a way, but those are not ideal for voice AI because we are aiming for, say one or two seconds, even lower latency, but for the entire conversation, there are a lot of steps. For example, how to detect the language, how to do the ASR, how to do voice isolation and get to the original sentence or text people ask and send it to LLM. How to detect the intent, how to call the right API, how to generate the response, and also that response should be valid, passing all the guardrails, preventing some prompt injection or some other patterns or text.

In the end, you have to generate an audio. It is a very long step, so if we use frameworks such as Claude Code or Python or LangChain or LangGraph, the latency can be a big issue, and also sometimes it is hard to inject a certain logic without forking their framework. We are being very careful.

Again, you mentioned this term over-engineering. We do not want to over-engineer ourselves, but for many enterprise customers, they do want to see all the possibilities to fine-tune every single step. For example, for the ASR part, you can rely on an engine to convert audio to text, but if your use cases involve a lot of special terms, you might want to somehow almost prompt this ASR engine to know, hey, this is the right spelling, this is the right term, even that term is not so common. There are so many opportunities we need to add extra control, extra configurations. This gives us a good reason to maintain our own harness framework.

Also, to quickly answer your question, the other one is, every agent, we do not have to rebuild a harness framework or even we do not have to build a new version of the harness framework. The harness framework is the same. For example, our harness framework supports, we call it an umbrella agent and a sub-agent. Essentially, it is almost like a state machine.

You have the global settings regarding what is the high-level goal and what is the guardrail, what should be said, what should not be said, but for every single sub-agent, it is almost like a topic or a state. For example, if the person wanted to change their address, change address can be a sub-agent, but if they wanted to cancel a credit card or add a new credit card, that can be the other sub-agent. The person can make just a single call and get multiple things done, and this is a multi-agent framework we support, but it is not the multi-agent in other scenarios. For example, in Claude Code, you might spin off multiple sub-agents to do things in parallel, do the research, and merge the results.

This is not so common in the voice AI scenario, because voice AI scenario, especially for the customer support, the AI don't need to be so creative. They also don't need to come up with multiple options for a user to choose. If you use Claude Code, it's very common, it will try different things and ask you, "Hey, can I ask you a few questions?" "Here are three options. Which one do you want to choose?"

For the case of customer experience, it is almost like agentic workflow. There's a lot of deterministic logic. If you wanted to change your flight or if you want to cancel your booking, usually there's only one answer. It should follow the best practice. It should call the right API. Our harness framework is very emphasized on the low latency, calling the right API, minimize the latency, and then also minimize the hallucination, anything like that. In the end, say last week we have an airline AI agent. This week, if we have an AI agent for insurance, essentially, we just build a set of YAML file as the configuration about the entire business logic, some of the prompt engineering, and also together with the Python-based tool call.

This can be packaged as a Docker image. That YAML file together with the Docker image will be deployed to our platform and leveraging almost the same harness framework. Again, we don't have to create a different version of the harness framework. In many cases, because we are fast, we just have one version of the harness framework. This can be improved every week.

---

Client 00:16:16
**One of the challenges** you mentioned was about reliable tool call. I'm understanding now that your path and your framework is quite deterministic. Can you maybe explain **what's challenging about** reliably calling the right tool? What have you tried to solve this challenge?


©2026, AlphaSense, Inc. All Rights Reserved. AlphaSense is a service mark of AlphaSense, Inc. All other trademarks mentioned belong to their respective owners.

Expert 00:16:32

Sure. Maybe because those are frontier models, they do iterate quite fast. Back to maybe one year ago, when we use GPT-4o mini or 4.1, **it is always a problem** to call the right tools, no matter you put how many tools in the system. Again, there's a lot of best practice. As a demo, you can always have some logic just written in English in the prompt. In the prompt, it says, if the user is over 20 years old, you should call this API. If the user is between 20-15, call this API. You can just use whatever markdown or natural language to give this instruction to the LLM to call certain tools. Those tools are registered in the system prompt.

Imagine if you have 10 or even 100 different if else in the prompt, then LLM can easily get confused. They might skip a certain step or they may confuse two tool calls with similar names or slightly different parameters. Those are very common, no matter you are using OpenAI, Anthropic, or other models. In general, the better or the newer model can do better instruction following. For example, **GPT 5.1 certainly is better than 4.1**. In some cases, maybe 5.4 is better than 5.1, but there also can be some regressions. We cannot really trust the benchmark or blog from those vendors.

We have to do the test by ourselves, making sure this is right. Again, you cannot put too many things in the prompt. The prompt should be as short as possible. Also, you cannot really put too many tools in the system. If you can just get things done using 10 tools, do not register 100 different tools because LLM essentially is a token generation tools. When you have, say, more than 20 different tools, **they might lose certain attention**, they might get confused. Our system is designed to minimize those confusion and also have a set of best practice. For example, we do not recommend our engineer, especially FDE, to write very long prompts. For example, we have this sub-agents design, as I mentioned earlier. You should create multiple sub-agents.

Each sub-agent can have their own prompt. They even can have different tool call and different models even. Every sub-agent, it's almost like divide and conquer? You break this into a smaller problem, don't mess up everything in a single agent or something like that.

---

Client 00:19:21

Makes sense. Can you also walk me through your eval setup? How do you make sure that your harness or that your agent as a whole is not just silently degrading in production?

Expert 00:19:31

Test eval is such a big topic in our system. In many cases, if we spent one month to build the AI agent, we might spend two or three or even four months on the test eval. There's a lot of design in our system regarding test eval. For example, there's some we call binary requirements. For example, at the beginning of the call, the AI agent have to disclose they are an AI agent, this is either yes or no. If someone mentioning they did not need to talk to an AI agent, and if they mention this for twice or three times, we have to transfer to a human agent without further trying to explain. There's certain rules or business logic we have to follow. We call it a binary requirement.

We can run various simulation to making sure no matter the simulation is manual or automated simulation, making sure those requirement is followed. We also have other test eval which is not always black or white. It can be a score. How friendly this conversation is, a thing like that. We can run the evaluation to see whether it is following our best practice. Also, there's a bunch of knowledge base related test evals. What's the commonly asked questions, what's the best answer for certain things, and whether the agents answer those questions in the expected ways. All those test eval is a big investment in our end. We even have these synthetic customers.

Essentially, based on our human-to-human conversation, because as I mentioned earlier, Cresta also do the software for the human agents, and this already running there for five or six years, even before people use AI agent. **We already save quite a lot of our conversation for real human guest talk to their human agents and all those conversations as a text-based format in our system, then we can run some machine learning tasks such as clustering or do some kind of anomaly** detection. We can figure out if this is airline use cases, then what is the top 10 typical customers? Are they care about the membership? Are they about to change the flight? Every single synthetic customer is backed by real conversations.

We can use this to generate more simulated visitors, so you don't have to make the phone call or you don't have to type in the text box. We can leverage AI to simulate those typical customers and let AI talk to the AI and we also have this LLM evaluator to measure whether our main AI agent is answering things in the expected way. There's all those simulation, all those evaluator AI and internal, there's a lot of tools around improve or ensure the quality of our AI agents.

---

Client 00:22:28

I also want to double click on cost optimization. I think this is a topic that we're seeing more and more in the industry. Are **you seeing pressure** to have a bigger ROI and ensure cost optimization for your agentic workflows and just your agents that you push to prod? If so, how do you approach this? Do you go more towards the model, towards the harness to see different ways that you can ensure cost optimization for the agent? What are the steps here to ensure that?



©2026, AlphaSense, Inc. All Rights Reserved. AlphaSense is a service mark of AlphaSense, Inc. All other trademarks mentioned belong to their respective owners.

Expert 00:22:58

Cost optimization is a big topic because we do not want to share exactly the number of tokens we are using for each use cases and also the model. We do not want to be a company to resell the token. Our goal is to making sure our enterprise customer, they can use AI agents to get their business need, either **it is cost reduction** or improve the customer satisfaction rate. Usually we charge our AI agent by the usage. Primarily, it is the number of minutes, each year, for all the human agent and the AI agent handle phone calls. The minutes are one key or common way. We also have provided other, for example, outcome-based charging. For example, how many tickets are resolved or if this is to collect say credit cards, then how many credit cards are collected?

---

Client 00:23:52

My question is more with regards to cost of running agents. Not cost for your end customer necessarily. I am just talking about the cost of actually running your agents, their harness plus the model. Are you at all concerned about optimizing as much as possible that cost of running them? If so, what are the mechanics that you are using to ensuring that?

Expert 00:24:16

I try to answer that from the high level, but again, because our charge model is not really to resell the token, so **we have a strong motivation** to almost save as much as possible so that we can get more margin. For example, if we can use a cheaper model to get things done, we do not want to use a more expensive model. We be very careful, not always use the latest model, for example, GPT-5.4.

For a long time, we just use GPT-4o mini or GPT-5.1. That is our part also regarding voice. For example, ElevenLabs is much more expensive than Cartesia and Deepgram. We do prefer using Cartesia as a default voice engine and also fall back to Deepgram, but if there is a certain very important deal and the customer really care about the voice or even the brand, maybe we can use ElevenLabs. We do have a little bit flexibility to choose which model we use or even which version we use. Also, we use systems such as Langfuse to provide a lot of detailed tracking, tracing on every single step. It is not just about the voice latency, but also the cost in every single step.

As I mentioned earlier, there are so many different models, different step in every single voice conversation, and we needed to making sure we do not really burn too much token on the main response. We do not burn too much token on the guardrail or even on the test eval. Every single LLM interaction can cost certain money. We will be very careful to understand, what is the main contributor, and how we can further save money. Even the other topic I want to mention quickly, that is about the caching and the context management. That's also a very tricky, but also key.

---

Client 00:26:09

Makes sense. We have heard a little bit about that as well, which I am curious as well, just going back to some of the points you mentioned, I want to make sure I understood you correctly. Do you also use open source models when applicable, instead of just using maybe older versions of frontier labs? If so, what are the use cases for which you are able to use an open source model?

Expert 00:26:34

We use open source model or we even fine-tune based on open source model, mainly for some very specific use cases. For example, summarization or sentiment analysis. In the case of our call center, when we end the call, we need to give a summary of the entire conversation and put in the system so that next time a human can review or the other agent can review.

That summarization don't really need a very powerful agent. In many cases, we just use our, I guess it's Mistral, or we fine-tune a model like that, or similar thing for whatever semantic analysis, but for the main AI agents, AIM, we use Frontier, so we don't want to use a small model because many of the use cases require a quite sophisticated model, so it can be all kind of different conversations human can talk to the AI agents, so using a small model or open source model can have the risk the AI don't give them a natural response or cannot make the right tool call or cannot summarize the knowledge base in a proper way.

Also there's some other factor, for example, if we use models such as DeepSeek or Minimax, there are some kind of a feedback or even pushback from the customer. Customers, even they rely on us to be the solution, but they want to know which model we are using, so using OpenAI and Anthropic will be the best choice.

---

Client 00:27:59

Do you see maybe American-based labs that are open source, actually maybe in the short term, starting to build open source models that are at par with what the Frontier Labs are doing? The reason I am saying American-based is **I am assuming the pushback** you're getting is because these models are, like DeepSeek is Chinese-based. Are you seeing the level of open source in America essentially equate to what you see in the frontier labs anytime soon? Would you be open to essentially transferring over to open source models if that were the case?



©2026, AlphaSense, Inc. All Rights Reserved. AlphaSense is a service mark of AlphaSense, Inc. All other trademarks mentioned belong to their respective owners.

Expert 00:28:30

Yeah, I don't think there's many promising open source model from U.S.-based lab. For example, in open source software, if we don't care too much about geopolitics, then Qwen is very popular. Even Kimi, DeepSeek. Again, the reason why we use Mistral, because it's based on France. It is relatively safe or less arguable, but I don't see many ready-to-use open source model just from U.S.

Also, as I mentioned earlier, even if you check our Cresta blog, we even view the model called Ocean One. This is the model we train by ourselves. Again, how we do realize that is using the frontier model is, today, this is almost a widely agreed pattern, use a frontier model together with some prompt management is the easiest way to get things done instead of training your own model, because training is expensive, and even the model you're training, it cannot answer all the questions at one time.

---

Client 00:29:32

I think it is a good time to ask about what a company in this space is actually doing. I would like to get your reaction on that as well, and any of the key questions, concerns, that may come on your side. Essentially, what the company does is their developer tool, their dev-facing end-to-end platform to enable devs to essentially deploy agents in their organization and provide a single control plane that gives them visibility into every agent that is running across the org. Cost per task, **error rates**, model performance, the actual agent performance as well, compliance audit trails as well, and what is interesting as well with that company is there is the ability to swap the underlying model to an open source one, or even adjust guardrails without having to redeploy the agent.

Essentially, they centralize everything into one platform to deploy and have visibility into those agents. Some core features that they have are workflows to capture eval datasets and train open source models. They have in-prod A/B testing or A/B/C/D. However, I think you can go up to five different testing scenarios for different agents. Let us say you have a specific use case, and you want to test different models, different harnesses. You can test up to five different agents at once and figure out which one is the most efficient based on your cost, latency, and accuracy matrix that you are working in. They have actual context-aware open source model routing.

You can describe the use case that you are looking for, and essentially, they can automatically figure out which open source model could be the right use case for this, fine-tune it, and then run an agent on it. Lastly, they have the control center with, as I mentioned, the per agent cost and ROI visibility on your entire fleet of deployed agents in your organization. What is your reaction overall to a company that does that? Essentially, an end-to-end platform for developers to really centralize the entire deployment of agents, but also the visibility post-deployment of your fleet.

Expert 00:31:36

Based on what you said, this is a very ambitious goal. I am not sure how ready they are to make the promise to check all the boxes. Again, for example, many of the test eval and visibility, today we rely on Langfuse. Langfuse can also do some of the test eval parts. If this is also part of this company's goal to, for example, based on your use case, to suggest which model comes closer to solve the problem, even provide some guiding regarding how to train it, it is just almost too good to be true. It is very hard to understand the subtle difference between different models. In many cases, it depends on the model itself.

---

Client 00:32:21

Maybe the four core features that they have, what if we rank them? The four ones that I mentioned, the first one is a little bit Langfuse style, to your point. Capturing workflows for evaluation datasets and then training open source models on those datasets. The second feature that they have is this A/B testing for production models. Essentially, the way you can visualize it is you look at your screen, you have four different agents running simultaneously in testing. You have a bunch of different models, different harnesses for each. All with the same underlying goal and use case. Of course, they take into consideration tool calling, guardrails, security, data residency. This is the observability A/B testing for your different agents to figure out which one essentially fulfills the criteria that you are looking for the most. The third one is context-aware open source model routing.

Essentially, based on natural language prompting, it figures out which open source model could be the best to route to for your specific use case. What is interesting with that as well, I do want to mention, is they say they have an AI **that continuously crawls and scrapes** all the new open source models that get released, for example, on Hugging Face. What they can do is they continue contesting with these models and figure out if there is a new addition you can essentially swap your model with, if there is a new one that essentially can do the job better. The last one is that control center with the entire ROI visibility, token cost per agent. Essentially, just the overall control layer for your fleet of agents. Those are the four features. Is there any one that sticks out if you were to rank them?

Expert 00:34:07

I will say that number three, to monitor what is the model available on the market and try to route your workload to the right model, that part, in my view, is just not so realistic or practical. Also, as a company building a lot of AI agents, we have to be very careful regarding almost the pattern. We do not want to change the behavior too randomly, and also we need to pay attention to the caching or other things. I cannot imagine, based on different prompts or different use cases, you actively route to different models. In that case, the caching rate will be very low, and you might be running your workload on some almost random models.



©2026, AlphaSense, Inc. All Rights Reserved. AlphaSense is a service mark of AlphaSense, Inc. All other trademarks mentioned belong to their respective owners.

You don't really have the full control which one we'll be using, and the message or the response from the AI agent can be more or less different. For example, if this is from xAI Grok, maybe it will be more straightforward, and maybe using OpenAI will be more polite. The tone will be quite different. The number three, I do have many questions there. For the A/B test, that's really very important. That's also something quest that we do ourself out the box. We do need to create some control group or test to measure, say, which way gets better response from the customer, or even if we can choose, say, this is using GPT 5.1, this is a 5.2 or 5.3, which model give us better customer success rate. A/B test or those control group is very common.

If this new company can provide a very easy way and this very sophisticated way to do this, that'll be great. Also, the other thing that is for us to work on a lot of the voice AI, but **we still struggle a lot** regarding how to test the voice. For example, if you just run a text-based AI agent, then you just check the text response to know whether this is right or not. If it is voice AI, then it is very hard to use AI to measure whether the other voice is natural or not, whether there's some unnatural high pitch or talk too slow or pronounce certain things in an interesting way. For example, even the same word, it can be pronounced in the U.S. way, in the U.K. way, in the India way.

It might end up with same text, but **many things can go wrong with the voice**. My point is, I don't think there's a good solution out there to do the voice-based test, not mentioning some of the how to handle the voice interruption, how to control the speed, stuff like that. If someone can really provide a very good voice AI testing evaluation, that'll be great. I don't think that's in the four items in your list.

---

Client 00:36:57

Thank you for your time. This has been very helpful, have a great rest of your day.



©2026, AlphaSense, Inc. All Rights Reserved. AlphaSense is a service mark of AlphaSense, Inc. All other trademarks mentioned belong to their respective owners.